Machine Learning Based Analog Circuit Fault Diagnosis
PDF 24 (Final) — Chapter 31: Complete Project Integration
The Entire Project From Start to Finish | The Complete 10-15 Minute Viva Script

PART 1 — The Entire Project From Start to Finish

Chapter 31 — Complete Project Integration Part 1 — The Entire Project From Start to Finish Before Starting Congratulations. You have now studied every script individually. But here's something important. A professor will never ask only "What does Day 13 do?" Instead, they will ask Explain your entire project. This chapter teaches you how the entire project works as one complete system. Think of this chapter as the bird's-eye view.

One Sentence Summary

Professor asks Explain your project in one sentence.

Perfect Answer

This project automatically detects faults in analog circuits by generating simulated circuit data, extracting meaningful signal features, training an optimized machine learning model, and combining it with engineering knowledge through a topology-aware decision rule for reliable fault diagnosis.

The Big Picture

Your entire project can be represented using one diagram. LTspice Circuit

Circuit Simulation

Waveform Generation

Dataset Creation

Noise Generation

Feature Extraction (17 Features)

Feature Dataset

StandardScaler — Optimized SVM.

Confusion Matrix Analysis

DC Level Improvement

Topology Rule

Dashboard

Fault Diagnosis

This is your complete project. Everything fits into this pipeline.

Stage 1 — Circuit Design

We begin inside LTspice. You designed multiple circuits. Example RC RLC Sallen-Key RC Ladder Each circuit was simulated under multiple conditions. Question. Why simulate? Because collecting real laboratory data for hundreds of fault conditions would require huge amounts of time, equipment, and money. Simulation provides a safe, repeatable, and scalable alternative.

PROFESSOR ASKS
Why did you use LTspice instead of real hardware?
EXCELLENT ANSWER

LTspice allows controlled simulation of multiple fault conditions with repeatable results. It enables rapid dataset generation without requiring expensive laboratory equipment while still producing physically meaningful circuit responses.

Stage 2 — Fault Injection

Each circuit was simulated under multiple fault conditions. Examples Normal

Open Circuit

Short Circuit

Drift Noise Every simulation produced one waveform. Question. Why multiple faults? Because

Machine Learning

must learn how each fault changes the circuit response.

Stage 3 — Waveform Processing

Each simulation generated many samples. Question. Can

Machine Learning

use waveforms of different lengths? No. Therefore, every waveform was resampled to exactly 200 points. Now every sample has the same size.

Stage 4 — Dataset Creation

Now every waveform becomes one row inside the dataset. Example Sample ID Circuit Fault v0 v1. v199 This becomes the master dataset.

Stage 5 — Noise Generation

Next, your project creates realistic Gaussian noise. Question. Why? Because real measurements always contain noise. Training only on ideal signals would reduce real-world performance. Therefore, a new Noise fault class was added.

Stage 6 — Feature Extraction

Now comes the heart of the project. Every waveform is converted into 17 meaningful features. Examples Peak RMS

Rise Time

Settling Time

FFT Features

Signal Energy

DC Level Instead of 200 voltage values,

Machine Learning

receives 17 informative descriptors.

PROFESSOR ASKS

Why use feature extraction?

EXCELLENT ANSWER

Feature extraction reduces dimensionality while preserving physically meaningful information about the circuit response. This improves interpretability, computational efficiency, and classification performance.

Stage 7 — Feature Dataset

The project creates a second dataset. Instead of

200 Samples

it contains

17 Features

+

Fault Label

This dataset is used for training.

Stage 8 — Machine Learning

The dataset is divided into Training and Testing sets. **Training** — Learns. **Testing** — Evaluates. Then StandardScaler normalizes the features. Finally, the SVM learns the fault patterns.

Stage 9 — Hyperparameter Optimization

Instead of using default settings, GridSearchCV searches for the best combination of C Gamma Kernel The RBF kernel becomes the best-performing classifier.

Stage 10 — Error Analysis

Now comes the turning point. Instead of celebrating the accuracy, you investigate the mistakes. The confusion matrix reveals that RC_Open is repeatedly misclassified.

Stage 11 — Engineering Improvements

First, you add DC Level. This captures steady-state voltage. Then, you introduce the

Topology Rule,

which combines

Machine Learning

with engineering knowledge. Now the project becomes a Hybrid AI System.

Stage 12 — Deployment

Finally, everything is integrated into a dashboard. The user simply uploads a waveform. The software does everything else.

Complete Data Flow

Let's follow one waveform. **LTspice** — CSV.

200 Samples

17 Features

Scaling — Optimized SVM. Prediction

Topology Rule

Final Diagnosis

That is the journey of every sample.

Why This Project Is Strong

Notice what makes your project different. It is not just Machine Learning. It combines Electronics +

Signal Processing

+ Statistics +

Machine Learning

+

Software Engineering

+

Engineering Knowledge

Very few student projects combine so many areas. What Skills Does This Demonstrate? By completing this project, you demonstrate knowledge of Electronics RC Circuits RLC Circuits Filters Fault Behaviour Signal Processing Waveform Analysis FFT Noise Modelling Feature Extraction Machine Learning Classification SVM Hyperparameter Tuning Cross Validation Confusion Matrix Python NumPy Pandas Scikit-Learn Matplotlib Pickle GUI Programming Software Engineering Modular Design Deployment Error Handling GUI Development Revision Notes Remember the complete project. **Simulation** — Dataset. Noise

Feature Extraction

Machine Learning
Optimization

Error Analysis

Engineering Improvements
Deployment Memory Trick Remember this sentence.

"Simulate, Extract, Learn, Improve, Deploy."
Those five words summarize your entire project.

KNOWLEDGE CHECK

Level 1

- What is the overall goal of the project?
- Why use LTspice?
- Why extract features?
- Why use SVM?
- Why build a dashboard?

Level 2

- Explain the complete data flow.
- Why is feature engineering important?
- Why is hyperparameter tuning performed?
- Why analyze the confusion matrix?
- Why combine ML with engineering knowledge?

Professor Level

- Explain your complete project architecture.
- Describe how data flows from LTspice simulation to dashboard prediction.
- What are the major engineering contributions in your project?
- Why is this project multidisciplinary?
- If you had another six months, what improvements would you make?
- End of Chapter 31 (Part 1)
- Congratulations.
- You can now explain your entire project from start to finish in a structured way.
- This is exactly the overview you'll use at the beginning of a viva, interview, or project demonstration.

Author's Note The next continuation will be Chapter 31 (Part 2) — The Complete 10–15 Minute Viva Script. This will include: A polished introduction. Step-by-step explanation exactly as you should speak. Smooth transitions between topics. Natural answers to follow-up questions. A professor-level presentation that sounds confident without being overly complicated. This is the script you can memorize and deliver almost verbatim during your project viva.

PART 2 — The Complete 10-15 Minute Viva Script

Chapter 31 — Complete Project Integration Part 2 — The Complete 10–15 Minute Viva Script Before Starting This chapter is different. Until now, I taught you the project. Now, I'll teach you how to present it. If tomorrow your professor says, "Explain your complete project." This is almost exactly how you should answer. Don't try to memorize every word. Instead, understand the flow. Complete Viva Script Good Morning Sir Good morning Sir. My project is titled "Machine Learning-Based Analog Circuit Fault Diagnosis Using Signal Processing and Hybrid AI." The objective of this project is to automatically identify faults in analog electronic circuits by combining circuit simulation, signal processing, machine learning, and engineering knowledge into one intelligent diagnostic system.

Problem Statement

Every electronic circuit can develop faults during manufacturing, operation, or aging. Examples include Open circuits Short circuits Component drift Measurement noise Traditionally, fault diagnosis requires manual testing using oscilloscopes and multimeters, which is time-consuming and depends heavily on expert knowledge. The goal of this project is to automate that process. Why Machine Learning? Instead of writing separate rules for every fault, I wanted the system to learn fault patterns directly from data. Machine learning can recognize complex relationships between waveform characteristics and fault conditions, making diagnosis faster and more scalable.

Stage 1 — Circuit Simulation

I first designed multiple analog circuits using LTspice. These include RC Circuit RLC Circuit Sallen-Key Filter RC Ladder Network Each circuit was simulated under different fault conditions, such as Normal, Open, Short, Drift, and later Noise. Every simulation generated a transient waveform. Why LTspice? LTspice allows controlled, repeatable, and low-cost generation of large datasets. Generating hundreds of real laboratory faults would require significant time, equipment, and cost. Simulation provides physically meaningful data much more efficiently.

Stage 2 — Dataset Generation

Each waveform was exported to CSV format. Since every waveform contained different numbers of samples, I resampled every waveform to exactly 200 points. This ensures that all samples have the same size, which is necessary for machine learning. The resulting dataset contains metadata, fault labels, and 200 voltage values for each sample.

Stage 3 — Noise Generation

Initially, the dataset contained only ideal signals. However, real electronic measurements always contain noise. Therefore, I generated additional Gaussian noise samples using a fixed 25 dB Signal-to-Noise Ratio. This created a realistic Noise fault class and improved the robustness of the model.

Stage 4 — Feature Extraction

Instead of training the model using all 200 voltage samples, I extracted 17 meaningful features. These include Peak Voltage RMS Mean Standard Deviation Rise Time Settling Time FFT Features Signal Energy DC Level These features summarize both time-domain and frequency-domain behavior

```
while reducing
```

the dimensionality of the problem. Why Feature Extraction? Feature extraction reduces computational complexity, improves interpretability, and allows the classifier to learn using physically meaningful signal characteristics instead of raw voltage values.

Stage 5 — Machine Learning

The extracted features were divided into training and testing datasets. Before training, all features were normalized using StandardScaler because SVM is sensitive to feature scale. I trained a baseline

Support Vector Machine
classifier and evaluated its performance using Accuracy,

Classification Report,
and Confusion Matrix.

Stage 6 — Hyperparameter Optimization
Instead of using default SVM settings, I optimized the classifier using GridSearchCV with 5-fold Cross Validation. I compared Linear, Polynomial, and RBF kernels. The RBF kernel produced the highest classification accuracy, so it became the final model.

Stage 7 — Error Analysis
Rather than stopping after obtaining good accuracy, I analyzed the confusion matrix. I observed that RC_Open was repeatedly misclassified. Instead of changing the machine learning algorithm, I investigated why those errors were occurring.

Stage 8 — Feature Engineering
I compared the extracted features of RC_Normal and RC_Open. I found that the feature set did not explicitly capture the steady-state voltage of the waveform. To solve this, I introduced a new feature called DC Level, which calculates the average of the last 20 waveform samples. After retraining, the confusion reduced significantly, confirming that the new feature captured previously missing information.

Stage 9 — Hybrid AI
The final improvement was the Topology Rule. Instead of relying only on machine learning, I combined the SVM prediction with engineering knowledge about different circuit topologies. The rule acts as a post-processing verification layer, making the system more reliable and more explainable.

Stage 10 — Dashboard
Finally, I developed a Python GUI that integrates the entire pipeline. The user simply uploads a waveform, and the dashboard automatically extracts features, scales them, predicts the fault, applies the Topology Rule, and displays the final diagnosis. This transforms the project from a research prototype into a usable software application.

Final Contributions
The main contributions of my project are Automatic dataset generation using LTspice. Extraction of 17 meaningful signal features. Optimization of an SVM classifier using GridSearchCV. Evidence-based feature engineering through DC Level. Development of a Hybrid AI system using the Topology Rule. Deployment through an interactive dashboard.

Future Scope
Future improvements could include Real hardware validation Deep Learning models Real-time oscilloscope integration Cloud-based fault diagnosis FPGA implementation Support for additional circuit topologies Conclusion In conclusion, this project demonstrates that combining signal processing, machine learning, and engineering knowledge can produce an accurate, interpretable, and practical fault diagnosis system for analog electronic circuits. Thank you.

If Professor Interrupts
These are the most common interruptions.

PROFESSOR ASKS

Why SVM? Answer Because the dataset is relatively small, feature-based, and contains nonlinear decision boundaries. SVM provides excellent performance for such structured datasets while maintaining good generalization. Professor Why not CNN? Answer CNNs require much larger datasets and typically learn features directly from raw signals. Since our dataset is moderate in size and we already extracted physically meaningful features, SVM was more appropriate. Professor Why FFT? Answer Some faults change the frequency content of the waveform rather than only the voltage levels. FFT captures this hidden information through frequency-domain features. Professor Why GridSearchCV? Answer To systematically optimize SVM hyperparameters using cross-validation instead of relying on default values or manual trial-and-error. Professor Why DC Level? Answer Because confusion matrix analysis revealed repeated RC_Open misclassification. DC Level explicitly captures the steady-state voltage that was missing from the original feature set. Professor Why Topology Rule? Answer Because machine learning alone cannot incorporate circuit-domain knowledge. The Topology Rule combines statistical prediction with engineering reasoning to improve reliability and explainability. Professor What is the biggest contribution? Answer The biggest contribution is not simply achieving good accuracy. It is the engineering workflow of identifying systematic errors through confusion matrix analysis, designing the DC Level feature to address those errors, and finally integrating machine learning with circuit knowledge through the Topology Rule. End of Chapter 31 (Part 2) Congratulations. At this point, your Project Bible covers: Every source file. Every algorithm. Every feature. Every engineering decision. The complete software pipeline. The complete 10–15 minute viva presentation. This gives you a coherent story you can present from introduction to conclusion while also being ready for detailed follow-up questions.